Chapter 5: Backend Application Setup

Welcome back! In our previous chapter, we learned all about the "User Authentication System" on the backend – how it securely manages your user accounts, verifies your identity, and protects your private data. But for any of that to actually *work*, our backend application itself needs to be up and running!

This chapter is all about setting up that foundational "engine room" for our backend server.

What Problem Are We Solving?

Imagine our entire interview-ai-yt application as a building. The frontend is the beautiful lobby and user-facing rooms. The AI service is a specialized research lab. The authentication system is the security office. But before any of these can function, the **building itself needs to be constructed, powered, and have its core systems (like plumbing and electricity) in place.**

The "Backend Application Setup" is precisely that:

1. **Starting the Server:** It's like turning on the main power switch for our application.
2. **Connecting to the Database:** This is laying the foundation and connecting to a water supply so our application can store and retrieve data (like your user accounts and interview reports).
3. **Setting up Core Tools:** This includes ensuring the server understands different types of incoming data (like JSON or file uploads), allowing our frontend to communicate with it, and handling cookies for authentication.
4. **Organizing Routes:** It's like setting up a central directory that directs incoming requests (e.g., "login," "generate report") to the correct processing units.

Essentially, this is the starting point that brings all backend services online and makes them ready to receive requests from our frontend.

Key Concepts: The Backend's Core Infrastructure

To get our backend server running smoothly, we rely on a few crucial components:

1. **Node.js & Express.js:** Our backend is built with Node.js, which is a way to run JavaScript code outside of a web browser. **Express.js** is like a robust framework (a set of tools and rules) that

makes building web applications with Node.js much easier. It helps us handle incoming requests and send back responses.

2. **Environment Variables (dotenv):** Our application needs secret keys (like JWT_SECRET for authentication) and connection strings (like our database address MONGO_URI). We don't want to hardcode these directly in our code, especially if they change or are sensitive. dotenv is a library that helps us load these "environment variables" from a special .env file, keeping them secure and flexible.

3. **Database Connection (Mongoose):** We store our application's data (users, reports, etc.) in a MongoDB database. **Mongoose** is a library that makes it easy for our Node.js application to talk to MongoDB, save data, find data, and update data.

4. Middleware (Traffic Cops & Translators): Imagine every request coming into our server as a car. Middleware functions are like "traffic cops" or "translators" that process these cars before they reach their final destination.

    1. **express.json():** This middleware helps our server understand incoming data that's formatted as JSON (a common way to send data over the web).
    2. **cookie-parser():** This middleware helps our server read and write cookies, which are crucial for our User Authentication System to remember who is logged in.
    3. **cors(): CORS (Cross-Origin Resource Sharing)** is a security feature in web browsers. Our frontend runs on http://localhost:5173, and our backend on http://localhost:3000. These are different "origins." cors middleware tells the browser that it's okay for our frontend to talk to our backend, preventing security errors.

5. **Routing:** This is how our server knows where to send different requests. For example, a request to /api/auth/login should go to our authentication logic, while a request to /api/interview/ should go to our AI report generation logic. Routing maps specific URLs to specific functions in our code.

How Our Backend Application Starts Up

As a developer, the main way you interact with this setup is by simply starting the server. This triggers a chain of events that brings everything online.

Here's how you would typically start our backend application:

```
# In your terminal, navigate to the 'Backend' folder
cd Backend

# Install all the necessary tools and libraries (dependencies)
npm install

# Start the server in development mode
npm run dev
```

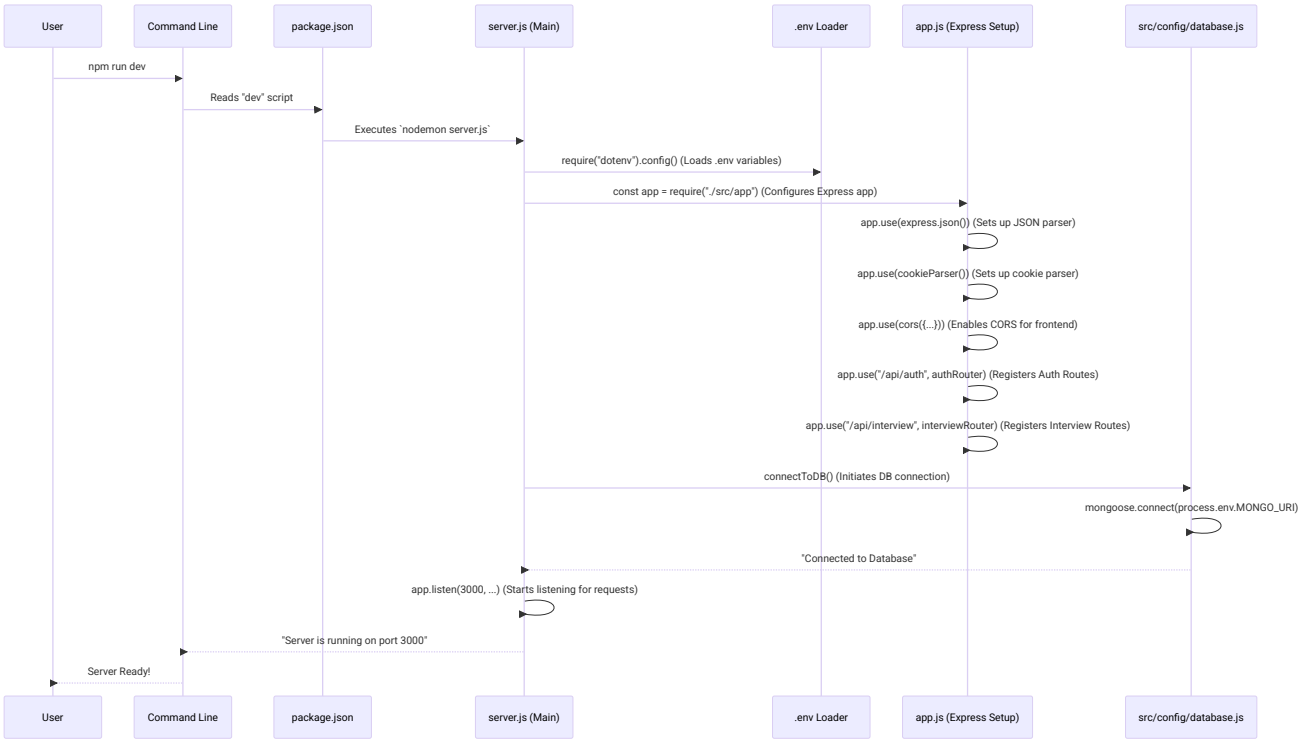
Explanation:

1. cd Backend changes your directory to where our backend code lives.
2. npm install reads the package.json file and downloads all the listed "dependencies" (like Express, Mongoose, dotenv) that our application needs to run.
3. npm run dev executes a command defined in package.json that typically starts server.js using nodemon. nodemon is a helpful tool that automatically restarts our server whenever we make changes to the code, saving us time during development.

Once you run npm run dev, you'll see messages in your terminal like "Connected to Database" and "Server is running on port 3000". This tells you that our backend application has successfully performed its setup tasks and is now ready to receive requests from the frontend!

Under the Hood: The Startup Sequence

Let's trace the steps our backend takes from when you type npm run dev until it's fully ready to serve requests.



Now, let's look at the core code that orchestrates this setup.

1. The Project's "Recipe Book" (Backend/package.json)

This file is like a recipe book for our project. It lists all the ingredients (dependencies) and instructions (scripts) to run our application.

```
// Backend/package.json (simplified)
{
  "name": "yt-genai",
  "version": "1.0.0",
  "scripts": {
    "dev": "npx nodemon server.js" // Our command to start the server!
  },
  "dependencies": {
    "@google/genai": "^1.42.0",
    "bcryptjs": "^3.0.3",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.2.1", // For database connection
    "multer": "^2.0.2",
    "pdf-parse": "^2.4.5",
    "puppeteer": "^24.37.5",
    "zod": "^3.25.76",
    "zod-to-json-schema": "^3.25.1"
  }
}
```

Explanation:
The scripts section defines commands like npm run dev. When you run this, it executes npx nodemon server.js, telling nodemon to watch and run our server.js file. The dependencies list shows all the powerful libraries our backend uses, like express for building the web server, mongoose for talking to the database, dotenv for environment variables, cors for cross-origin communication, and cookie-parser for handling cookies.

2. The Main "Switchboard" (Backend/server.js)

This is the main entry point for our backend application. It's like the central switchboard that turns on all the essential systems.

```
// Backend/server.js
require("dotenv").config(); // 1. Load environment variables first!
const app = require("./src/app"); // 2. Get our Express application setup
```

```
const connectToDB = require("./src/config/database"); // 3. Get the database connector

connectToDB(); // Connect to the database

app.listen(3000, () => { // 4. Start the server and listen for incoming requests
   console.log("Server is running on port 3000");
});
```

Explanation:

1. require("dotenv").config() is crucial: it loads all our secret keys and configurations from the .env file *before* the rest of the app tries to use them.
2. const app = require("./src/app") imports the core Express application configuration (which we'll see next).
3. const connectToDB = require("./src/config/database") imports our function to connect to the database.
4. connectToDB() then runs to establish the database connection.
5. Finally, app.listen(3000, ...) tells our Express application to start listening for web requests on port 3000. This is the point where our backend is officially "online" and ready to receive traffic.

3. The Application "Blueprint" (Backend/src/app.js)

This file contains the core setup for our Express application, defining the foundational middleware and routes. Think of it as the architect's blueprint for the building's internal layout.

```
// Backend/src/app.js
const express = require("express");
const cookieParser = require("cookie-parser");
const cors = require("cors");

const app = express(); // Initialize our Express application

// ----- Essential Middleware (Traffic Cops & Translators) -----
app.use(express.json()); // Allows server to understand JSON data in requests
app.use(cookieParser()); // Allows server to read/write cookies for authentication
app.use(cors({ // Allows our frontend (http://localhost:5173) to talk to this backend
   origin: "http://localhost:5173",
   credentials: true // Important: allows cookies to be sent across origins
}));

// ----- Routes (Road Signs) -----
// Import our authentication routes (for login/register)
const authRouter = require("./routes/auth.routes");
```

```
// Import our interview routes (for AI reports, resume generation)
const interviewRouter = require("./routes/interview.routes");

// Tell Express which router handles which base URL
app.use("/api/auth", authRouter); // All /api/auth/* requests go to authRouter
app.use("/api/interview", interviewRouter); // All /api/interview/* requests go to interviewRouter

module.exports = app; // Export the configured app for server.js to use
```

Explanation:

1. const app = express() creates an instance of our Express application.
2. app.use(express.json()) makes sure our server can automatically understand and process JSON data sent in the body of incoming requests (like when you send your login details).
3. app.use(cookieParser()) enables our server to easily read and set HTTP cookies. This is vital for our User Authentication System to manage user sessions.
4. app.use(cors({...})) is a critical security setting. It explicitly tells web browsers that our frontend (running on http://localhost:5173) is allowed to make requests to this backend (running on http://localhost:3000), and importantly, credentials: true allows those requests to include authentication cookies.
5. Finally, we require our different "routers" (authRouter, interviewRouter) and then use app.use("/api/auth", authRouter) to tell Express: "Any request starting with /api/auth should be handled by the logic defined in authRouter." This keeps our application organized.

4. The Database "Foundation" (Backend/src/config/database.js)

This file is solely responsible for establishing the connection to our MongoDB database.

```
// Backend/src/config/database.js
const mongoose = require("mongoose");

async function connectToDB() {
  try {
    // Connect using the URI from our .env file
    await mongoose.connect(process.env.MONGO_URI);
    console.log("Connected to Database");
  }
  catch (err) {
    console.error("Database connection error:", err);
  }
}

module.exports = connectToDB;
```

Explanation:
The connectToDB function uses mongoose.connect() to establish a connection to our database. Notice process.env.MONGO_URI – this is where the dotenv magic comes in! Our database connection string is loaded securely from the .env file, not hardcoded into the application. If the connection is successful, it logs a message; otherwise, it catches and logs any errors.

Conclusion

In this chapter, we've laid the essential groundwork for our interview-ai-yt backend application. We saw how server.js acts as the main orchestrator, loading environment variables, setting up the Express app with necessary middleware (express.json, cookie-parser, cors), connecting to our database via Mongoose, and finally bringing the server online to listen for requests.

This setup is the silent, crucial foundation upon which all our AI services, user authentication, and data management are built. Now that our server can connect to the database, how do we actually tell the database what kind of information we want to store (like users and interview reports)? That's what we'll explore in our next chapter!
